

Tabular Reinforcement Learning

Temporal differences

Olivier Sigaud

Sorbonne Université
<http://people.isir.upmc.fr/sigaud>



Temporal difference mechanisms

Reinforcement learning

- ▶ In Dynamic Programming (planning), T and r are given
- ▶ Reinforcement learning goal: build π^* without knowing T and r
- ▶ **Model-free approach**: build π^* without estimating T nor r
- ▶ **Actor-critic approach**: special case of model-free
- ▶ **Model-based approach**: build a model of T and r and use it to improve the policy

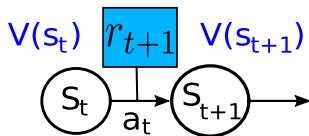
Incremental estimation

- ▶ Estimating the average immediate (stochastic) reward in a state s
- ▶ $E_k(s) = (r_1 + r_2 + \dots + r_k)/k$
- ▶ $E_{k+1}(s) = (r_1 + r_2 + \dots + r_k + r_{k+1})/(k+1)$
- ▶ Thus $E_{k+1}(s) = k/(k+1)E_k(s) + r_{k+1}/(k+1)$
- ▶ Or $E_{k+1}(s) = (k+1)/(k+1)E_k(s) - E_k(s)/(k+1) + r_{k+1}/(k+1)$
- ▶ Or $E_{k+1}(s) = E_k(s) + 1/(k+1)[r_{k+1} - E_k(s)]$
- ▶ Still needs to store k
- ▶ Can be approximated as

$$E_{k+1}(s) = E_k(s) + \alpha[r_{k+1} - E_k(s)] \quad (1)$$

- ▶ Converges to the true average (slower or faster depending on α) without storing anything
- ▶ Equation (1) is everywhere in reinforcement learning

Temporal Difference error



- ▶ The goal of TD methods is to estimate the value function $V(s)$
- ▶ If estimations $V(s_t)$ and $V(s_{t+1})$ were exact, we would get $V(s_t) = r_{t+1} + \gamma V(s_{t+1})$
- ▶ The approximation error is

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \quad (2)$$

- ▶ δ_t measures the error between $V(s_t)$ and the value it should have given $r_{t+1} + \gamma V(s_{t+1})$
- ▶ If $\delta_t > 0$, $V(s_t)$ is under-evaluated, otherwise it is over-evaluated
- ▶ $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$ should decrease the error (value propagation)

Temporal Difference update rule

$$\begin{aligned}
 (1) \quad & E_{k+1}(s) = E_k(s) + \alpha[r_{k+1} - E_k(s)] \\
 (2) \quad & \delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \\
 (3) \quad & V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]
 \end{aligned}$$

Diagram illustrating the Temporal Difference update rule:

The diagram shows three states: s_{t-1} , s_t , and s_{t+1} . Above each state is a set of colored bars representing features. Arrows indicate the sequence of states: $s_{t-1} \rightarrow s_t \xrightarrow{a_t} s_{t+1}$. Blue arrows show the backward propagation of value: $V(s_{t-1}) \leftarrow V(s_t) \leftarrow V(s_{t+1})$.

- Combines two estimation processes:
 - incremental estimation (1)
 - value propagation from $V(s_{t+1})$ to $V(s_t)$ (2)

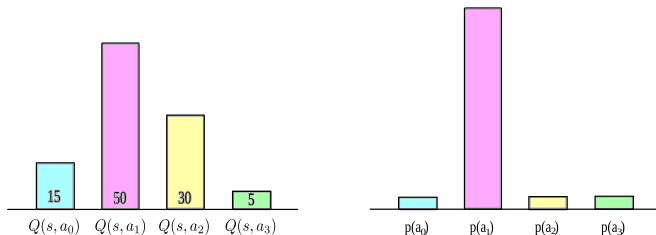
The Policy evaluation algorithm: TD(0)

- ▶ An agent performs a sequence
 $s_0, a_0, r_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, r_{t+2}, \dots$
- ▶ Performs local Temporal Difference updates from s_t, s_{t+1} and r_{t+1}
- ▶ Proved in 1994 provided ϵ -greedy exploration
- ▶ Note: updates can be performed in any order



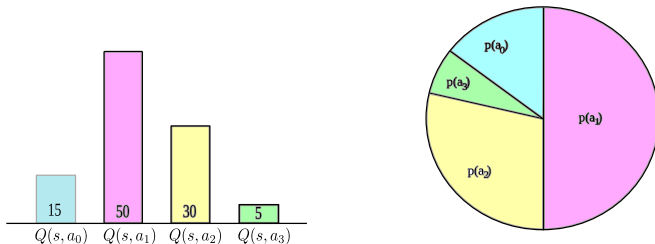
Dayan, P. & Sejnowski, T. (1994). TD(lambda) converges with probability 1. *Machine Learning*, 14(3):295–301.

ϵ -greedy exploration



- ▶ Choose the best action with a high probability, other actions at random with low probability
- ▶ Same properties as random search
- ▶ Every state-action pair will be enough visited under an infinite horizon
- ▶ Useful for convergence proofs

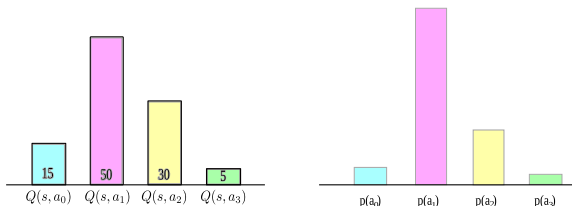
Roulette wheel



$$p(a_i) = \frac{Q(s, a_i)}{\sum_j Q(s, a_j)}$$

- The probability of choosing each action is proportional to its value

Softmax exploration



$$p(a_i) = \frac{e^{\frac{Q(s, a_i)}{\beta}}}{\sum_j e^{\frac{Q(s, a_j)}{\beta}}}$$

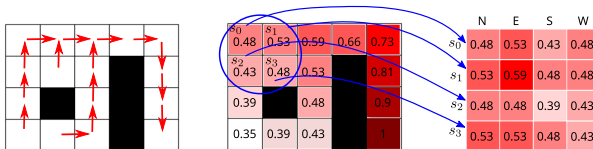
- The parameter β is called the temperature
- If $\beta \rightarrow \infty$, all actions have the same probability $\rightarrow$ random choice
- If $\beta \rightarrow 0$, increase contrast between values
- More used in computational neurosciences
- In machine learning RL, one often uses $\tau = \frac{1}{\beta}$, i.e. $p(a_i) = \frac{e^{\tau Q(s, a_i)}}{\sum_j e^{\tau Q(s, a_j)}}$
- Meta-learning: tune β dynamically (exploration/exploitation)

TD(0): limitation

- ▶ TD(0) evaluates $V(s)$
- ▶ One cannot infer $\pi(s)$ from $V(s)$ without knowing T : one must know which a leads to the best $V(s')$
- ▶ Three solutions:
 - ▶ Q-LEARNING, SARSA: Work with $Q(s, a)$ rather than $V(s)$.
 - ▶ ACTOR-CRITIC methods: Simultaneously learn V and update π
 - ▶ DYNA: Learn a model of T : model-based (or indirect) reinforcement learning

RL algorithms

Value function and Action Value function



- ▶ The **value function** $V^\pi : S \rightarrow \mathbb{R}$ records the aggregation of reward on the long run for each state (following policy π). It is a **vector** with one entry per state
- ▶ The **action value function** $Q^\pi : S \times A \rightarrow \mathbb{R}$ records the aggregation of reward on the long run for doing each action in each state (and then following policy π). It is a **matrix** with one entry per state and per action

SARSA

- ▶ Reminder (TD): $V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$
- ▶ SARSA: For each observed $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$:
 $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$
- ▶ Policy: perform exploration (e.g. ϵ -greedy)
- ▶ One must know the action a_{t+1} , thus constrains exploration
- ▶ On-policy method: more complex convergence proof



Singh, S. P., Jaakkola, T., Littman, M. L., & Szepesvari, C. (2000). Convergence Results for Single-Step On-Policy Reinforcement Learning Algorithms. *Machine Learning*, 38(3):287–308.

SARSA: the algorithm

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

► Taken from Sutton & Barto, 2018

Q-LEARNING

- For each observed $(s_t, a_t, r_{t+1}, s_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_{a \in A} Q(s_{t+1}, a) - Q(s_t, a_t)]$$

- $\max_{a \in A} Q(s_{t+1}, a)$ instead of $Q(s_{t+1}, a_{t+1})$
- **Off-policy method**: no more need to know a_{t+1}
- Policy: perform exploration (e.g. ϵ -greedy)
- Convergence proven given infinite exploration



Watkins, C. J. C. H. (1989). *Learning with Delayed Rewards*. PhD thesis, Psychology Department, University of Cambridge, England.



Watkins, C. J. C. H. & Dayan, P. (1992) Q-learning. *Machine Learning*, 8:279–292

Q-LEARNING: the algorithm

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R , S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

► Taken from Sutton & Barto, 2018

Q-LEARNING: details of the algorithm

Algorithm 1: Q-LEARNING

```
1: obs = env.reset()
2: total_reward = 0
3: done = False
4: while not done do
5:   action = agent.get_action(obs, deterministic)
6:   next_obs, reward, terminated, truncated, _ = env.step(action)
7:    $Q[\text{obs}, \text{action}] = Q[\text{obs}, \text{action}] + \alpha [\text{reward} + \gamma \max_{a'} Q[\text{next\_obs}, a'] - Q[\text{obs}, \text{action}]]$ 
8:   done = terminated or truncated
9:   obs = next_obs
10:  total_reward += reward
11: end while
12: return total_reward
```

- ▶ Note the gym(nasium) interface
- ▶ Why $(1 - \text{terminated})$?

Episode termination

- ▶ **Terminated:** the episode finishes due to success or failure
- ▶ **Truncated:** the episode finishes because the time limit is reached
- ▶ $1 - \text{Terminated} = 0$ if Terminated is True, $= 1$ if Terminated is False
- ▶ $Q[\text{obs}, \text{action}] = Q[\text{obs}, \text{action}] + \alpha[\text{reward} + \gamma \text{np.max}(Q[\text{next_obs}, :]) - Q[\text{obs}, \text{action}]]$
- ▶ Is equivalent to:

Algorithm 2: Bellman backup

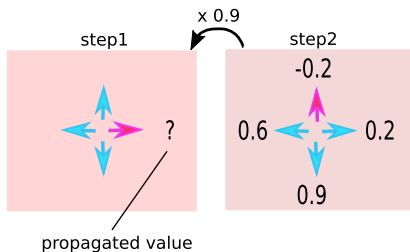
```
1: if Terminated then  
2:    $Q[\text{obs}, \text{action}] = Q[\text{obs}, \text{action}] + \alpha[\text{reward} - Q[\text{obs}, \text{action}]]$   
3: else  
4:    $Q[\text{obs}, \text{action}] = Q[\text{obs}, \text{action}] + \alpha[\text{reward} + \gamma \text{np.max}(Q[\text{next\_obs}, :]) - Q[\text{obs}, \text{action}]]$   
5: end if
```

- ▶ If the episode is truncated, we should still bootstrap, as if the episode had continued



Pardo, F., Tavakoli, A., Levdiĭk, V., and Kormushev, P. (2018) Time limits in reinforcement learning. In *International Conference on Machine Learning*, pages 4045–4054. PMLR

Difference between Q-LEARNING and SARSA



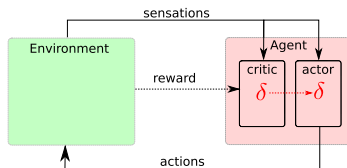
- Consider an agent taking the two pink actions
- With Q-LEARNING, the propagated value ? is $\gamma \arg\max_a Q(s_{t+1}, a)$, thus $0.9 \times 0.9 = 0.81$
- With SARSA, it is $\gamma Q(s_{t+1}, a_{t+1})$, thus $0.9 \times -0.2 = -0.18$

Q-LEARNING in practice

 0.0	 0.0	 0.0	 0.0	 0.0
 0.0	 0.0	 0.0		 0.0
 0.0		 0.0		 0.0
 0.0	 0.0	 0.0		0.0

- ▶ Build a $\text{states} \times \text{actions}$ table (*Q-Table*, eventually incremental)
- ▶ Initialise it (randomly or with 0 is not a good choice)
- ▶ Apply update equation after each action
- ▶ Problem: it is (very) slow

Actor-critic: Naive design



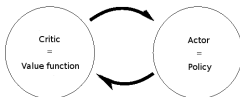
- ▶ Discrete states and actions, stochastic policy
- ▶ An update in the critic generates a local update in the actor
- ▶ Critic: compute δ and update $V(s)$ with $V_{k+1}(s) \leftarrow V_k(s) + \alpha_k \delta_k$
- ▶ Actor: $P_{k+1}^\pi(a|s) \leftarrow P_k^\pi(a|s) + \alpha_k \delta_k$
- ▶ NB: no need for a max over actions
- ▶ NB2: one must know how to “draw” an action from a probabilistic policy (not straightforward for continuous actions)



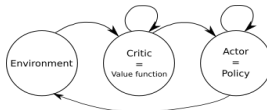
Williams, R. J. and Baird, L. (1990) A mathematical analysis of actor-critic architectures for learning optimal controls through incremental dynamic programming. In *Proceedings of the Sixth Yale Workshop on Adaptive and Learning Systems*, pages 96–101

Dynamic Programming and Actor-Critic

Policy Iteration



Actor-Critic



- ▶ In both PI and AC, the architecture contains a representation of the value function (the critic) and the policy (the actor)
- ▶ In PI, the MDP (T and r) is known
- ▶ PI alternates two stages:
 1. Policy evaluation: update $(V(s))$ or $(Q(s, a))$ given the current policy
 2. Policy improvement: follow the value gradient
- ▶ In AC, T and r are unknown and **not represented** (model-free)
- ▶ Information from the environment generates updates in the critic, then in the actor

From $Q(s, a)$ to Actor-Critic

state / action	a_0	a_1	a_2	a_3
e_0	0.66	0.88*	0.81	0.73
e_1	0.73	0.63	0.9*	0.43
e_2	0.73	0.9	0.95*	0.73
e_3	0.81	0.9	1.0*	0.81
e_4	0.81	1.0*	0.81	0.9
e_5	0.9	1.0*	0.0	0.9

state	chosen action
e_0	a_1
e_1	a_2
e_2	a_2
e_3	a_2
e_4	a_1
e_5	a_1

- ▶ Given a Q - Table, one must determine the max at each step
- ▶ This becomes expensive if there are numerous actions
- ▶ Store the best value for each state
- ▶ Update the max by just comparing the changed value and the max
- ▶ No more maximum over actions (only in one case)
- ▶ Storing the max is equivalent to storing the policy
- ▶ Update the policy as a function of value updates

Any question?



Send mail to: Olivier.Sigaud@isir.upmc.fr



Dayan, P. and Sejnowski, T. (1994).
TD(λ) converges with probability 1.
Machine Learning, 14(3):295–301.



Pardo, F., Tavakoli, A., Levdić, V., and Kormushev, P. (2018).
Time limits in reinforcement learning.
In *International Conference on Machine Learning*, pages 4045–4054. PMLR.



Singh, S. P., Jaakkola, T., Littman, M. L., and Szepesvári, C. (2000).
Convergence results for single-step on-policy reinforcement-learning algorithms.
Machine learning, 38(3):287–308.



Velentzas, G., Tzafestas, C., and Khamassi, M. (2017).
Bio-inspired meta-learning for active exploration during non-stationary multi-armed bandit tasks.
In *2017 Intelligent Systems Conference (IntelliSys)*, pages 661–669. IEEE.



Watkins, C. J. C. H. (1989).
Learning with Delayed Rewards.
PhD thesis, Psychology Department, University of Cambridge, England.



Watkins, C. J. C. H. and Dayan, P. (1992).
Q-learning.
Machine Learning, 8:279–292.



Williams, R. J. and Baird, L. (1990).
A mathematical analysis of actor-critic architectures for learning optimal controls through incremental dynamic programming.
In *Proceedings of the Sixth Yale Workshop on Adaptive and Learning Systems*, pages 96–101. Citeseer.